

DESIGN

Modelling

Diagrammi UML

Il team ha modellato il sistema utilizzando i diagrammi UML citati di seguito.

- Use case diagram : definisce i confini del sistema e le interazioni degli attori Utente ed Admin
- Class diagram : rappresenta la struttura del sistema e le interazioni tra le varie classi ed oggetti
- State Machine diagram: modellato il flusso dell'accesso e della prenotazione
- Sequence diagram: rappresenta flusso di messaggi dal tentativo di prenotazione alla sua conferma
- Communication diagram: rappresenta flusso di messaggi dal tentativo di prenotazione alla sua conferma in maniera alternativa al sequence diagram
- Activity Diagram: descrive il flusso completo dell'utente all'interno dell'applicazione, dall'accesso sino alla conferma della prenotazione
- Component Diagram: illustra l'architettura del sistema e le dipendenze tra le varie componenti, sia interne che esterne
- Package Diagram: visualizza l'organizzazione dei pacchetti java, senza creare cicli per garantire un codice pulito e manutenibile

I diagrammi presenti in questo documento sono ritrovabili all'interno della cartella del codice (find-n-charge/diagrammiUML) ed in versione pdf nella cartella "diagrammi pdf".

Software Architecture

Stile architetturale

Stile architetturale: il software utilizza un architettura a strati (layered) per garantire la separazione delle responsabilità e facilitare la manutenibilità.

Il sistema è diviso in tre layer principali:

- Presentation Layer (Views): Gestione dell'interfaccia utente (Vaadin).
- Business Layer (Service): Logica applicativa e validazione.
- Data Access Layer (Interface + FirebaseService): Gestione dati.

Componenti e connettori

Componenti principali: oggetti, Firebase, Client Browser, Spring, backend Vaadin

Connettori principali: chiamate a Firebase, chiamate Spring (injection), chiamate HTTPS.

Gestione delle dipendenze

La gestione delle dipendenze è avvenuta tramite la loro aggiunta al POM tramite Maven.

Sono state integrate differenti librerie tra le quali:

- Dipendenze Spring – per il corretto funzionamento di SpringBoot e Tomcat
- Dipendenze ZXing – per la generazione dei QR code
- Dipendenze BCrypt – per la gestione di password criptate
- ApexCharts – per grafici e statistiche
- Vaadin – per completa gestione frontend

Software Design

Design patterns

Il progetto sfrutta Design Pattern quali:

- Singleton Pattern: utilizzo di @Service e @Repository garantisce la creazione di una sola istanza di quella classe
- Adapter Pattern: utilizzo di interfacce permette di cambiare database in qualunque momento
- Observer Pattern: utilizzo di addClickListener per frontend e ValueEventListener per backend

Metriche del codice

È stata effettuata un'analisi del codice e sono emersi i seguenti dati (aggiornato al 01/02/2026)

Categoria	Metrica	Valore	Note
Volume	ELOC (Estimated Lines of Code)	3.505	Valore consono per progetto universitario.
	Numero Unità	51	Distribuite su 12 pacchetti (Media 4,25 classi/package).
Complessità	McCabe (Complessità Ciclomantica)	1,25	Codice lineare se il valore di complessità è minore di 10.
	Pollution	0,51	<i>Vedi grafico sottostante.</i> Valore influenzato dall'utilizzo di Vaadin.
Struttura	Tangled Libraries	0%	Assenza totale di cicli tra i pacchetti.
	Metrica di Martin (D)	0,37	Progetto bilanciato nella "Main Sequence" (Stabilità vs Astrazione).
Metriche OO	CBO (Coupling Between Objects)	2,27	Basso accoppiamento: le classi sono indipendenti e modulari.
	WMC (Weighted Methods per Class)	7,08	Complessità interna contenuta. Assenza di "God Class".
	DIT (Depth of Inheritance Tree)	0,94	Valore ridotto
	RFC (Response For a Class)	7,86	Numero limitato di metodi eseguiti per chiamata (Alta testabilità).

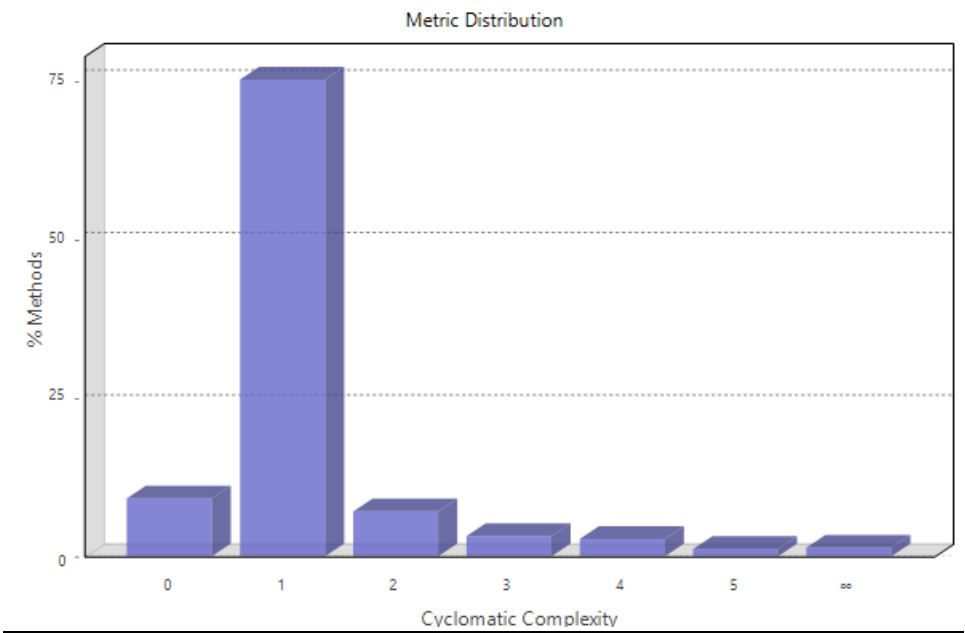
NB Il valore ridotto di complessità media è influenzato anche dalla presenza di getter e setter con complessità pari a 1.

Analizzando più specificatamente (vedi sotto) si ottiene comunque che la percentuale di metodi con complessità maggiore di 4 è solo il 2% dei metodi totali.

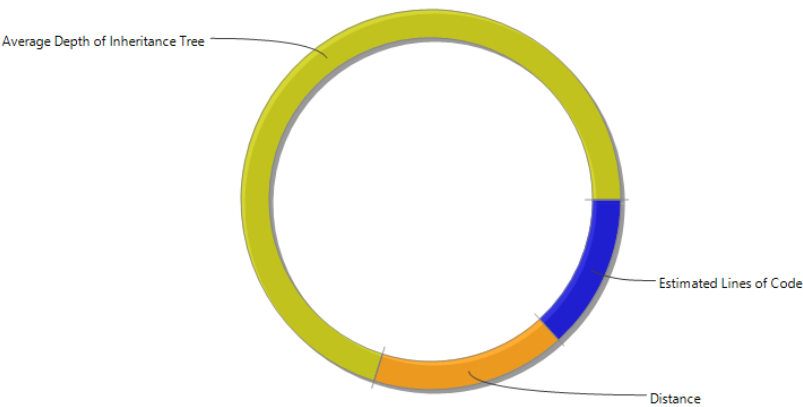
Di seguito le percentuali di metodi e le loro relative complessità.

Grafici e dati

Complessità metodi e percentuali:



Pollution:



Pollution: 0.51

Metriche presenti in tabella:

Category/Metric	Value
▼ Count	
Libraries	4
Packages	12
Units	51
Units / Package	4.25
Classes / Class	0
Methods / Class	5.66
Fields / Class	2.01
ELOC	3505
ELOC / Unit	68.73
▼ Complexity	
CC	1.25
Fat - Libraries	1
Fat - Packages	25
Fat - Units	123
Tangled - Libraries	0.00%
ACD - Library	8.33%
ACD - Package	26.52%
ACD - Unit	10.55%
▼ Robert C. Martin	
D	-0.3
D	0.37
▼ Chidamber & Kemerer	
WMC	7.08
DIT	0.94
NOC	0
CBO	2.27
RFC	7.86
LCOM	10.62